
Reproducible Training-Free Inference Acceleration on Pythia-70M

Ruoyu Fu
NLP Course Final Project
Code: GitHub repository

Abstract

Efficient inference is a practical bottleneck for autoregressive language models because attention computation and the KV cache grow with context length. This work studies training-free inference modifications on Pythia-70M: a post-hoc grouped-query attention (GQA) approximation and two KV cache compression strategies implemented with KVPress, Knorm and StreamingLLM. We evaluate exact teacher-forced perplexity, cached next-token perplexity, time to first token (TTFT), time per output token (TPOT), and throughput on WikiText-103 and a PG-19 sample. StreamingLLM-style KV compression improves WikiText throughput by 21.7–27.6% while preserving a reasonable cached-PPL score, whereas the post-hoc GQA approximation severely degrades perplexity and does not reliably accelerate inference. On PG-19, all compression variants slow down in our CPU-only runtime, showing that algorithmic cache reduction can be dominated by Python hook and gather overhead. The experiments can be reproduced with the scripts included in the repository.

1 Introduction

Autoregressive decoding repeatedly evaluates a transformer while maintaining a key-value (KV) cache for previous tokens. This cache avoids recomputing attention states, but its memory and attention cost scale with sequence length. Recent efficient-inference work therefore attacks the problem from several directions: using fewer KV heads (Ainslie et al., 2023), more IO-aware attention kernels (Dao et al., 2022), or compressing the KV cache during generation (Xiao et al., 2023; NVIDIA, 2024).

This project focuses on methods that can be reproduced without training a new model. We use Pythia-70M (Biderman et al., 2023), a small GPT-NeoX model, as the common backbone. The assignment asks for both quality and speed evaluation on datasets such as WikiText and PG-19, so we report two complementary quality metrics. First, exact perplexity is computed in the standard teacher-forced way and is not affected by KVPress, because KVPress acts on generation caches rather than full-context forward passes. Second, cached perplexity evaluates next-token prediction after a prefill pass whose KV cache may be compressed. This second metric is the appropriate quality signal for KVPress in this implementation.

The main contribution is a reproducible comparison under a CPU-only course environment. The results show that post-hoc GQA is not a reliable drop-in acceleration method for a model trained with standard multi-head attention, while KV cache compression can help on some contexts but is sensitive to runtime overhead.

33 2 Methods

34 **Baseline.** The baseline loads the local EleutherAI/pythia-70m checkpoint through Hugging
35 Face Transformers and uses the default scaled dot-product attention (SDPA) implementation. No
36 weights are changed.

37 **Post-hoc GQA approximation.** Pythia-70M uses 8 attention heads. In the GQA variant, we
38 compute the original query, key, and value states, average the key/value heads into 2 grouped KV
39 heads, then repeat the grouped KV states back to 8 heads before attention. This preserves tensor
40 compatibility with the pretrained checkpoint, but it is only an inference-time approximation; the
41 model was not trained under the grouped-KV constraint. We therefore expect possible quality
42 degradation.

43 **KVPress cache compression.** We evaluate two KVPress modes at compression ratio 0.5. Knorm
44 keeps tokens with larger key norm scores. StreamingLLM keeps attention-sink tokens and recent
45 tokens, following the observation that early sink tokens are important for stable long-context decoding
46 (Xiao et al., 2023). In both cases, compression is applied after the prefill stage via forward hooks on
47 GPT-NeoX attention layers. The implementation records per-layer cache lengths before and after
48 compression, e.g., 512 cached positions are compressed to 256.

49 **Metrics.** Exact PPL is computed over 4096 tokens using 512-token chunks. Cached PPL first
50 prefills 512 context tokens, optionally compresses the resulting cache, and then scores the next 256
51 ground-truth tokens autoregressively with the cache. Speed is measured with greedy generation for
52 64 new tokens at context lengths 128, 512, and 1024. We report TTFT, TPOT, and throughput. All
53 experiments in this report are single-run CPU measurements, so small differences should not be
54 over-interpreted.

55 3 Experimental setup

56 We evaluate on the test split of WikiText-103 (Merity et al., 2016) and on one PG-19 test sample
57 (Rae et al., 2020), matching the course requirement that PG-19 can be evaluated with a single long
58 sample. The local environment is CPU-only with PyTorch 2.6.0, Transformers 4.56.2, Datasets 3.6.0,
59 and KVPress 0.5.3. Since CUDA is unavailable, FlashAttention is not used as a main experimental
60 condition.

61 The exact reproduction command is:

```
62 cd finalproj  
63 .\scripts\run_matrix.ps1
```

64 This script runs baseline, GQA, KVPress-Knorm, and KVPress-StreamingLLM on both datasets and
65 regenerates comparison_quality.* and comparison_speed.*.

66 4 Results and discussion

67 Table 1 summarizes quality. The exact-PPL column should be used to compare baseline and GQA.
68 The cached-PPL column should be used to compare KVPress cache-compression behavior, because it
69 scores continuation tokens after compressed prefill. GQA causes a large degradation: WikiText exact
70 PPL rises from 63.72 to 2258.01, and PG-19 exact PPL rises from 33.57 to 816.69. This confirms
71 that converting a pretrained MHA checkpoint into grouped KV heads by simple averaging is not
72 quality-preserving.

73 The WikiText baseline has much lower cached PPL than full-window exact PPL. To check whether
74 this was a cache-scoring artifact, we recomputed ordinary teacher-forced PPL on the identical
75 continuation slice: the first 512 tokens were used as context and the next 256 tokens were scored.
76 The same-slice teacher-forced PPL was 13.088, while cached autoregressive PPL was 13.089. This
77 small difference indicates that cached scoring matches standard teacher forcing on the same tokens.
78 The gap from the 63.72 exact PPL is therefore caused mainly by the evaluation window: exact
79 PPL averages over 4095 tokens, whereas cached PPL uses a fixed local continuation. We therefore

Table 1: Quality metrics. Exact PPL is standard teacher-forced PPL; cached PPL scores 256 continuation tokens after a 512-token prefill cache.

Dataset	Method	Exact PPL	Cached PPL	KV events	Avg. comp.
WikiText	Baseline	63.72	13.09	0	–
WikiText	GQA-2KV	2258.01	3658.94	0	–
WikiText	KVPress-Knorm	63.72	52.44	42	0.50
WikiText	KVPress-StreamingLLM	63.72	61.30	42	0.50
PG-19	Baseline	33.57	29.60	0	–
PG-19	GQA-2KV	816.69	776.88	0	–
PG-19	KVPress-Knorm	33.57	31.76	42	0.50
PG-19	KVPress-StreamingLLM	33.57	29.49	42	0.50

Table 2: Throughput change relative to baseline. Positive values indicate faster generation.

Dataset	Method	128 ctx	512 ctx	1024 ctx
WikiText	GQA-2KV	-2.2%	-14.0%	-7.5%
WikiText	KVPress-Knorm	+4.2%	+2.0%	+1.8%
WikiText	KVPress-StreamingLLM	+26.7%	+21.7%	+27.6%
PG-19	GQA-2KV	-58.2%	-86.7%	-88.9%
PG-19	KVPress-Knorm	-81.6%	-87.7%	-87.3%
PG-19	KVPress-StreamingLLM	-84.9%	-84.7%	-86.5%

80 compare cached PPL only across methods on the same cached-PPL slice, not against exact PPL as an
81 absolute model-quality number.

82 Table 2 reports throughput relative to the baseline. On WikiText, KVPress-StreamingLLM gives the
83 clearest acceleration: throughput improves by 26.7%, 21.7%, and 27.6% at context lengths 128, 512,
84 and 1024, respectively. Knorm gives only small gains of 1.8–4.2%. GQA does not provide reliable
85 speedup and also has poor quality, so it is mainly a negative result.

86 The PG-19 speed results are negative: every non-baseline method is slower. This does not mean
87 KV compression is useless; rather, it shows that in this CPU implementation the overhead of Python
88 hooks, tensor gathering, and cache manipulation can exceed the attention savings. This is especially
89 plausible for Pythia-70M, where the model is small and the absolute attention workload is modest. A
90 GPU implementation with fused kernels and larger models would be a more favorable setting for
91 cache compression.

92 5 Conclusion

93 The most reproducible positive result is KVPress-StreamingLLM on WikiText, which improves
94 throughput by about 22–28% while explicitly compressing each prefill cache by 50%. Knorm gives
95 weaker speed gains. Post-hoc GQA is not successful: it sharply worsens perplexity and does not
96 consistently accelerate generation. Overall, the study supports a cautious conclusion: training-free
97 KV cache compression is a practical direction for inference acceleration, but its benefit depends on
98 implementation overhead, dataset/runtime behavior, and model size. Negative results are important
99 here because they prevent overstating a method that is theoretically appealing but not effective under
100 the actual course environment.

101 References

- 102 Biderman, S., Schoelkopf, H., Anthony, Q., Bradley, H., O’Brien, K., Hallahan, E., Khan, M. A., Purohit, S.,
103 Prashanth, U. S., Raff, E., Skowron, A., Sutawika, L., and Van Der Wal, O. Pythia: A suite for analyzing
104 large language models across training and scaling. *International Conference on Machine Learning*, 2023.
- 105 Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebron, F., and Sanghai, S. GQA: Training generalized
106 multi-query transformer models from multi-head checkpoints. *Empirical Methods in Natural Language
107 Processing*, 2023.

- 108 Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Re, C. FlashAttention: Fast and memory-efficient exact attention
109 with IO-awareness. *Advances in Neural Information Processing Systems*, 2022.
- 110 Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. Efficient streaming language models with attention sinks.
111 arXiv:2309.17453, 2023.
- 112 NVIDIA. KVPress: KV cache compression for efficient LLM inference. [https://github.com/NVIDIA/](https://github.com/NVIDIA/kvpress)
113 [kvpress](https://github.com/NVIDIA/kvpress), 2024.
- 114 Merity, S., Xiong, C., Bradbury, J., and Socher, R. Pointer sentinel mixture models. arXiv:1609.07843, 2016.
- 115 Rae, J. W., Potapenko, A., Jayakumar, S. M., and Lillicrap, T. P. Compressive transformers for long-range
116 sequence modelling. *International Conference on Learning Representations*, 2020.